Hochschule Fulda
University of Applied Sciences

Global Distributed Software Development/
Teamprojekt (AI5088)

“Master Team Project Fall 2024”

Way2Home

Group 5 (Local)

Team Lead: Romina Nikasa
romina.nikasa@informatik.hs-fulda.de

Milestone 4

(Dated: 02/06/2025)

Supervisor: Prof. Dr. Todtenhöfer, Rainer

1) Product Summary

Way2Home is an intuitive and secure web platform tailored to simplify the housing search for **Fulda University students**, with a special focus on the challenges faced by international students. Our team, consisting of Fulda University students who have firsthand experience navigating the rental market, has designed **Way2Home** to provide a reliable, user-friendly, and moderated housing marketplace. The platform streamlines the process of finding accommodations by offering advanced search filters, personalized listings, and a seamless communication system between landlords and students.

Way2Home includes the following key features:

- **User Registration:** Users can create accounts with specific roles and permissions, ensuring secure access to features.
- **User Login:** Users can log in using their verified email and password. Incorrect credentials will prompt an error message: *"Email or Password is not valid."*
- **User Profile Management:** Students can complete or update their profiles with contact information, profile photos, and short bios.
- **Basic Search**: Students can filter search results based on various key words, like price, place, type of apartment.
- **Advance Search:** Students can refine their housing searches using filters such as price, location, property type, pet friendliness, availability dates**.**
- **View Property Details:** Users can access **detailed property information**, including photos, descriptions, rental costs, and landlord details.
- **Chat Feature:** Direct messaging allows students and landlords to **communicate securely** for inquiries, negotiations, and finalizing rental agreements.
- **Add Property Listings:** Landlords can upload properties by providing essential details, including images, descriptions, rental terms, and amenities. Listings require **approval** before going live.
- **Update Listings:** Landlords can update property details,
- **Save Search Functionality**: Users can save their searches for quick access later.
- **Favorite Listings**: Students can bookmark listings to revisit easily.
- **Tenant Score Assessmen**t: Landlords can fill a pre-defined questionnaire to evaluate prospective tenants
- **Respond to Tenant Questionnaire**: Students can complete a landlord's questionnaire to improve their rental chances.
- **Media Upload for Listings**: Landlords can upload images and videos to showcase properties.

- **Multiple Property Management**: Landlords can manage multiple listings under a single account.
- **Reviewing and Approving Property Listings** – Moderators approve listings to ensure quality and compliance.

Target Audience

Way2Home is designed specifically for **Fulda University students**, including **international and local students** who face difficulties in finding affordable and reliable accommodations. By focusing on this niche community, we aim to create a **trusted housing ecosystem** that simplifies the rental process and fosters transparency between students and landlords.

Way2Home – Unique Selling Points (USPs)

1. **Verified Listings for Trust & Safety**
    - Every listing is reviewed and approved by a moderator before going live, significantly reducing the risk of fake listings and fraud.
2. **Tenant Score for Smarter Matching**
    - A unique **Tenant Score** is generated based on a questionnaire, helping landlords identify the most suitable tenants quickly and efficiently.

Marketing Channels for Way2Home

1. **Campus Outreach and Awareness Campaigns**
    - **Flyer Drops with QR Codes:** Distribute flyers with QR codes leading to Way2Home's website.
    - **Student Ambassadors:** Recruit students as brand ambassadors to spread awareness in dorms and common areas.
    - **Pop-up Events:** Organize fun events like a “Find Your Home Faster” challenge with prizes for engagement.
    - **Housing Info Booths:** Set up interactive booths on campus with live platform demos.
2. **Social Media Campaigns**
    - **Weekly Housing Spotlights:** Feature listings with unique benefits (e.g., best deals, pet-friendly homes).
    - **Instagram Reels & TikTok Videos:** Show real-life moving-in stories and “house tours” by students.
    - **#MyWay2Home Contest:** Encourage students to share their rental experiences for a chance to win vouchers.

    - **Live Q&A Sessions:** Host Instagram/Facebook Live events answering student housing questions.
3. **Collaborations with University Departments**
    - **Integration into Orientation Programs:** Introduce Way2Home during international student welcome sessions.
    - **Collaboration with Student Advisors:** Work with Fulda University's housing support team for student referrals.
4. **Referral Programs and Incentives**
    - **Invite & Earn:** Reward users who invite friends with discounts on featured listings or housing-related services.
    - **Landlord Discounts for Referrals:** Encourage landlords to list more properties by offering fee reductions for referrals.
    - **Exclusive Early Access:** Provide referrers with early access to new property listings.

2) Usability Test Plan

Major Function: Add Listing (For Landlords)

Test Objectives

The objective of this usability test is to assess the ease of use, efficiency, and satisfaction of the "Add Listing" feature. This feature enables landlords to post new property listings by entering details such as name, location, type, availability dates, and rental costs. The test aims to evaluate how effectively users can complete the process, ensuring a smooth and user-friendly experience that encourages continued engagement with the platform.

Test Background and Setup

System Setup: The system to be tested is a property rental platform accessible through a desktop/mobile web browser. Testers will need a stable internet connection and a modern browser (e.g., Chrome or Firefox). No additional tools are required for testing.

Starting Point: The tester begins on the landlord's homepage after logging into their account. They navigate to the "Listings" section from the navigation bar and click on the "Add Listing" button to access the feature.

Intended Users: The intended users are landlords who wish to post properties for rent on the platform. Test participants should have basic familiarity with online forms and web interfaces.

URL of the System: https://gdsd-new.norwayeast.cloudapp.azure.com

What is to Be Measured:

In this usability test, we want to measure user satisfaction with the "Add Listing" feature. We will evaluate this through a short questionnaire containing six Likert scale (1-5) questions, which will be given to users after they have completed the task of adding a listing.

Usability Task Description

Instructions for the Tester:

1. Login to your landlord account and navigate to the "Listings" section from the navigation bar.
2. Click on the "Add Listing" button to access the form.
3. Fill out the form with the required details.
4. Submit the listing by clicking the "Submit Listing" button.
5. After completing the task, fill out the provided Likert scale questionnaire to share your feedback.

Measuring Effectiveness

Testing Users Detail

1. Supply chain management student
2. International management student
3. International management student
4. Working Student in Logistics & Supply Chain Management
5. International management student

During usability testing for the **Add Listing** feature, users were asked to evaluate how well they could add a listing to the platform. The process was tested for clarity, ease of use, and how accurately users could submit the required information. The feature was considered effective if users could easily understand the steps and fill in all the necessary fields without confusion. Users were also asked to rate the clarity of error messages and whether the design of the page helped them navigate through the task smoothly. The effectiveness of the feature was measured based on whether users felt confident in submitting their listing and whether they found the overall process to be

straightforward. This testing provided valuable insights into how well the **Add Listing** feature works and if any improvements were needed to make it more effective for users.

Measuring Efficiency

The efficiency of the Add Listing feature is measured using a Likert scale, where users rate their experience based on ease of use, clarity, and design. If most users give high ratings, it means the feature is efficient, while lower scores indicate areas for improvement. An efficient feature allows users to add a listing smoothly, understand the process easily, and submit details without confusion. A well-designed layout and clear error messages help users complete the task quickly and confidently. Another key factor is the time taken to finish the process—if users feel it is quick and hassle-free, the feature is considered efficient. To analyze efficiency, we calculate the **average score** from all user responses; a **higher score suggests a positive experienc**e, while a **lower score highlights areas that need improvement**.

User Satisfaction Questionnaire

Testers will respond to the following statements on a 5-point Likert scale (1 - Strongly Disagree, 2-Disagree, 3-Neutral, 4-Agree, 5 = Strongly Agree).

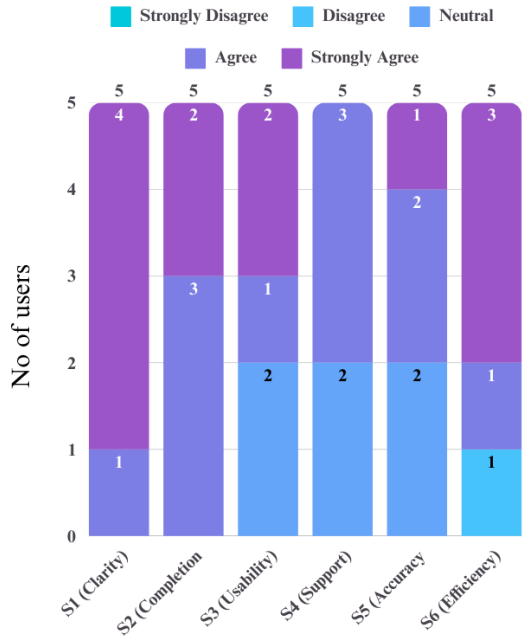
- ★ S1 (Clarity): "I found the process of adding a listing to be clear and easy to understand."
- ★ S2 (Completion): "I was able to fill in the required information and submit the listing with no confusion, ensuring completion."
- ★ S3 (Usability): "The layout and design of the 'Add Listing' page were intuitive, offering great usability."
- ★ S4 (Support): "The error messages, if any, were clear and provided helpful support in guiding me to resolve the issues."
- ★ S5 (Accuracy): "I felt confident that the information I entered was accurately captured by the system."
- ★ S6 (Efficiency): "The time it took to complete the 'Add Listing' task was reasonable and efficient."

How much do you agree with the following statements?

Usability Features	**Strongly Disagree**	**Disagre e**	**Neutral**	**Agree**	**Strongly Agree**
S1 (Clarity): I found the process of adding a listing straightforward and easy to				5	1,2,3,4

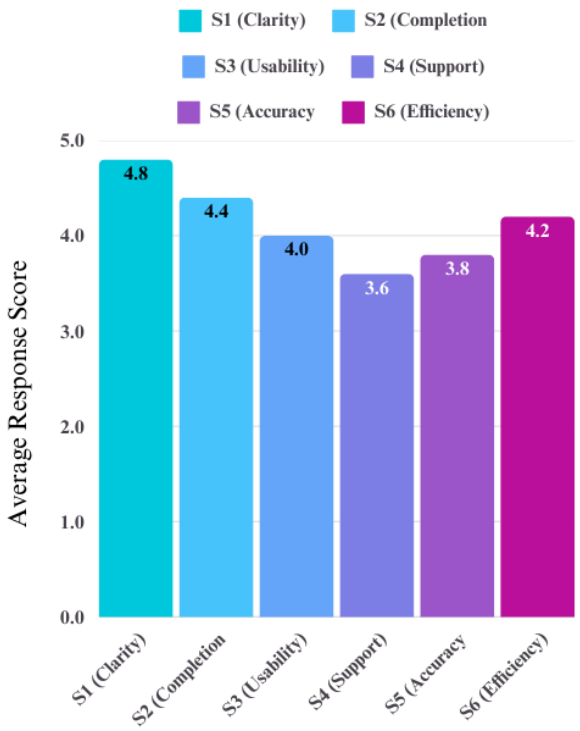
understand					
S2 (Completion): I was able to fill in the required information and submit the listing without confusion				1,4,5	2,3
S3 (Usability): The layout and design of the 'Add Listing' page were intuitive and user-friendly			3,4	2	1,5
S4 (Support): The error messages (if any) were clear and helpful in guiding me to resolve the issues			1,2	3,4,5	
S5 (Accuracy): I felt confident that the information I entered was accurately captured by the system			2,3	1,4	5
S6 (Efficiency): The time it took to complete the 'Add Listing' task was reasonable and efficient		5		4	1,2,3

User Satisfaction Graph



Efficiency Graph Based on Average Response Score

The average response scores show that most users had a positive experience with the "Add Listing" feature. Scores above 4.0 indicate that users found the process clear and efficient, with only slight room for improvement, particularly regarding error messages and system accuracy. The most liked statement was **S1 (Clarity)**: "I found the process of adding a listing straightforward and easy to understand," which had an average score of 4.8. The statement with the least average score was **S4 (Support)**: "The error messages (if any) were clear and helpful in guiding me to resolve the issues," with an average score of 3.6, still above neutral, indicating some room for improvement.



3) QA test plan

QA Test Plan for "Add Listing" Feature

Test Objectives

The objective is to validate the functionality of the "Add Listing" feature. This includes verifying that landlords can successfully add a new property listing, that the form inputs work as expected, and that the feature performs consistently across different browsers.

Hardware and Software Setup

- **Hardware**: Desktop/Laptop with a stable internet connection
- **Software**:
  - Operating Systems: Windows 10 or macOS
  - Browsers: Google Chrome (Version: 132.0.6834.160), Mozilla Firefox (Version:134.0.2)
- **URL**: https://gdsd-new.norwayeast.cloudapp.azure.com

Feature to be Tested

The "Add Listing" feature, which allows landlords to input property details (such as name, location, type, availability dates, and rental costs) and submit the listing for the moderator to review and approve.

Test #	Test Title	Test Description	Test Input	Expected Correct Output	Test Results (Chrome / Firefox)
1	Form Validation	Verify that all required fields are validated correctly.	Leave "Name", "Location", and "Cold Rent" fields empty; try to submit the form.	Error messages displayed for required fields: "Name is required", "Location is required", "Cold Rent is required".	Chrome: PASS Firefox: PASS

2	Successful Form Submission	Verify that the form submits successfully when all fields are filled correctly.	Input valid data into all fields, including uploading a media file and selecting the questionnaire option.	Listing is successfully submitted, and a confirmation message is displayed.	Chrome: PASS Firefox: PASS
3	Date Range Validation	Verify that "Availability From" cannot be after "Availability To".	Enter "02/15/2025" in "Availability From" and "02/10/2025" in "Availability To".	Error message displayed: "Availability From date cannot be after Availability To date".	Chrome: PASS Firefox: PASS
4	Media Upload Validation	Verify that the media upload field does not accept video files (e.g., .mp4).	Try uploading a video file (e.g., "video.mp4") and then upload an image file (e.g., "apartment.jpg").	Error message displayed for video file upload; successful upload for the image file.	Chrome: PASS Firefox: PASS
5	Checkbox Functionality	Verify that the "Are Pets Allowed?" checkbox is working correctly.	Check and uncheck the "Are Pets Allowed?" checkbox and submit the form.	Checkbox value (checked/unchecked) is accurately reflected in the submitted listing.	Chrome: PASS Firefox: PASS

Results of Testing

The tests were performed on both Google Chrome and Mozilla Firefox. All test cases passed successfully, confirming that the feature works as intended on both browsers.

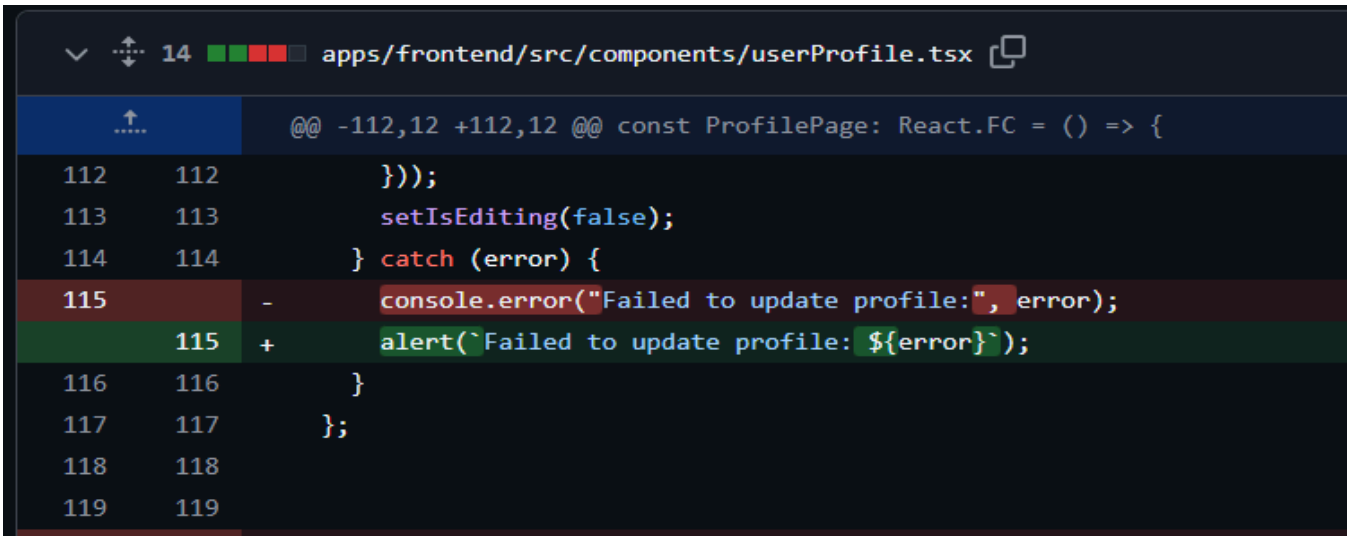
4) Code Reviews

Code Review #1

Update to the User Profile - Frontend Page
Code by Maira Shafi, reviewed by Anmoldeep
Links: [Pull request](), userProfile.tsx

Suggestion #1:
Code Before And PR Review Comment

Changes/Updated Code



Suggestion #2:

Code Before And PR Review Comment

fdai7674 commented yesterday

Team5-GDSD/apps/frontend/src/components/userProfile.tsx
Lines 316 to 335 in 6369caf

```
<button
onClick={handleSave}
className="w-full px-6 py-2 bg-black text-white border-2 border-black rounded-lg hover:bg-yellow-600 transition dur
>
Save
</button>
<button
onClick={() => setIsEditing(false)}
className="w-full px-6 py-2 bg-black text-white border-2 border-black rounded-lg hover:bg-yellow-600 transition dur
>
Cancel
```

The button styles are repeated multiple times. Consider using a styles object to manage reusable styles.

1

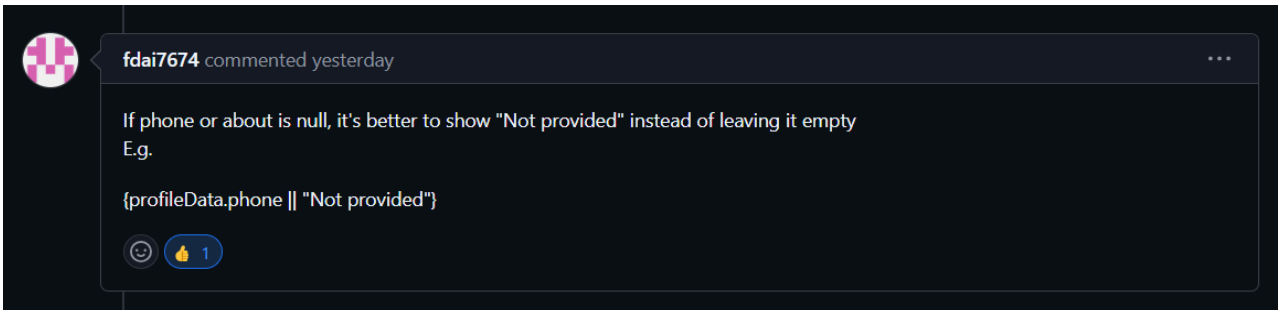
Changes/Updated Code

```
119  119
120       -
     120  + const buttonStyles = "w-full px-6 py-2 bg-black text-white border-2 border-black rounded-lg hover:bg-yellow-600 transition duration-300";
121  121
```

14 apps/frontend/src/components/userProfile.tsx

```
@@ -315,21 +315,21 @@ const ProfilePage: React.FC = () => {
315  315                <>
316  316                    <button
317  317                    onClick={handleSave}
318       -            className="w-full px-6 py-2 bg-black text-white border-2 border-black rounded-lg hover:bg-yellow-600 transition duration-300"
     318  +            className={buttonStyles}
319  319                    >
320  320                    Save
321  321                    </button>
322  322                    <button
323  323                    onClick={() => setIsEditing(false)}
324       -            className="w-full px-6 py-2 bg-black text-white border-2 border-black rounded-lg hover:bg-yellow-600 transition duration-300"
     324  +            className={buttonStyles}
325  325                    >
326  326                    Cancel
327  327                    </button>
328  328                </>
329  329                ) : (
330  330                <button
331  331                    onClick={() => setIsEditing(true)}
332       -            className="w-full px-6 py-2 bg-black text-white border-2 border-black rounded-lg hover:bg-yellow-600 transition duration-300"
     332  +            className={buttonStyles}
333  333                >
334  334                    Edit Profile
335  335                </button>
```

Suggestion #3:
Code Before And PR Review Comment



Changes/Updated Code

```
268  268                <div className="border-b pb-2">
269  269                  <div className="text-sm text-gray-500">Phone Number</div>
270  270                  <div className="text-lg font-medium text-gray-900">
271       -                {profileData.phone || "N/A"}
     271  +                {profileData.phone || "Not Provided"}
272  272                  </div>
273  273                </div>
274  274
@@ -301,7 +301,7 @@ const ProfilePage: React.FC = () => {
301  301                  <div className="border-b pb-2">
302  302                    <div className="text-sm text-gray-500">About</div>
303  303                    <div className="text-lg font-medium text-gray-900">
304       -                  {profileData.about}
     304  +                  {profileData.about || "Not Provided"}
305  305                    </div>
306  306                  </div>
307  307                )}
```

Code Review #2

Update to the chat notification system
Code by Tim Schröder, reviewed by Suhaila Kondappilly Aliyar
Links: Pull request, chat.router.ts (before & after), ChatPage.tsx (before & after)

Suggestion #1

ts4321 commented 2 days ago Author

Team5-GDSD/apps/backend/src/routers/chat.router.ts
Lines 31 to 38 in 455e515

```
31      const user = await DB.users.findOne({ id: userId })
32      if (!user) {
33          res.status(400).json({
34              success: false,
35              errors: `user with id ${user} not found in db`
36          })
37          return
38      }
```

> The error message is referencing the whole user instead of userId "errors: `user with id ${user} not found in db".

I have changed the message!

In the end, the code got removed due to the 2nd suggestion.

Suggestion #2

Team5-GDSD/apps/backend/src/routers/chat.router.ts
Lines 91 to 97 in 455e515

```
91      router.get(
92          "/:currentUser/:otherUser/:property",
93          [
94              param("currentUser").isString(),
95              param("otherUser").isString(),
96              param("property").isString()
97          ],
```

> The currentUserId can be fetched from the authenticateToken rather than passing in the URL. And it would be nice if the other parameters names exactly points to otherUserId and propertyId.

I have removed the user id from the urls and renamed the parameters.

Parts of the resulting code:

Team5-GDSD/apps/backend/src/routers/chat.router.ts
Lines 79 to 100 in 555bea5

```
79      router.get(
80          "/:otherUserId/:propertyId",
81          [
82              param("otherUserId").isString(),
83              param("propertyId").isString()
84          ],
85          async (req: Request, res: Response): Promise<void> => {
86              const errors = validationResult(req)
87              if (!errors.isEmpty()) {
88                  res.status(400).json({
89                      success: false,
90                      errors: errors.array()
91                  })
92                  return
93              }
94
95              const currentUserId = req.user!.userId
96              const {
97                  otherUserId,
98                  propertyId
99              } = req.params
100
```

Suggestion #3

Team5-GDSD/apps/backend/src/routers/chat.router.ts
Lines 114 to 129 in 455e515

```
114     const users = await DB.users.findAll({
115         where: {
116             id: {
117                 $in: [currentUserId!, otherUserId!]
118             }
119         }
120     })
121
122     const missingUsers = [currentUserId!, otherUserId!].filter(e => !users.map(e => e.id).includes(e))
123     if (missingUsers.length > 0) {
124         res.status(400).json({
125             success: false,
126             errors: missingUsers.map(e => `user ${e} not found in db`)
127         })
128         return
129     }
```

> Instead of fetching all users and checking for the userIds can cause more complexity if it is having a large user set. Isn't better to use findOne() for currentUserId and otherUserId existance and then throw the error if the array is not empty?

Not needed anymore since there will only be one user id now :)

Suggestion #4

Team5-GDSD/apps/frontend/src/components/ChatPage.tsx
Lines 71 to 83 in 455e515

```
71      const chatId = getChatId(otherUserId, propertyId)
72      if (!chats.has(chatId)) {
73        const { messages, otherUser, property, unreadCount } = await fetchChatMessages(currentUserId, otherUserId, propertyId)
74
75        chats.set(chatId, {
76          otherUser,
77          property,
78          messages,
79          unreadCount,
80          hasFetchedAllMessages: true
81        })
82        setChats(new Map(chats))
83      }
```

Just a small suggestion, instead of directly modifying chats it would be better to create a new chat map first and then update values to that and then call setChats(newChat). Also this code is used in multiple places and to avoid duplication, it can be wrapped to a function as well.

Good idea, I added a function for this.

Resulting code:

Team5-GDSD/apps/frontend/src/components/ChatPage.tsx
Lines 48 to 52 in 555bea5

```
48      const setChat = (chatId: string, data: ChatData) => {
49        const map = new Map(chats)
50        chats.set(chatId, data)
51        setChats(map)
52      }
```

Code Review #3

Implemented a new feature : Moderator Dashboard
Code by Romina Nikasa , reviewed by Maira Shafi

Feature/dashboard #43

Merged **maira-fulda** merged 6 commits into `feature/vertical-prototype` from `feature/dashboard` yesterday

Conversation 6 | Commits 6 | Checks 0 | Files changed 14

romiinik commented 3 days ago

Hi Maira,
I implemented a dashboard (insight page) for moderators.Both frontend and backend.
Can you please do the review?
Thanks

1

Romina Nikasa added 5 commits 3 days ago

implemented requests to get most-visited , most-favorited and most-ap…	399ab59
integrated requests with the frontend	ed59fc9
cleanup	4dfa18e
implemented cards for property status count and user count	404b145
added auth and few improvements	c421887

romiinik requested a review from **maira-fulda** 3 days ago

Suggestion #1

maira-fulda reviewed 2 days ago

View reviewed changes

`apps/frontend/src/components/moderator/Insights.css` Outdated

maira-fulda 2 days ago

Since we are using Tailwind CSS and not external CSS files in our project, it would be better to remove Insights.css and use Tailwind utility classes directly within insights.tsx to maintain consistency.

romiinik 2 days ago

Author

Thanks for the feedback @maira-fulda !
Removed Insights.css and converted all styles to Tailwind utility classes as suggested.

Putting this as an example of the changes i made :

```
<header className="bg-white shadow rounded-lg p-5 mb-8">
      <div className="max-w-4xl mx-auto text-center">
           <h1 className="text-2xl font-bold text-gray-800">Insights</h1>
           <p className="text-gray-600">Analyze property and user activity</p>
      </div>
</header>
```

Suggestion #2

maira-fulda reviewed 2 days ago

View reviewed changes

`apps/frontend/src/components/moderator/insights.tsx` Outdated

```
+
+     if (error) {
+         return <div>Error: {error}</div>;
+     }
```

maira-fulda 2 days ago

Instead of separate conditional return, use conditional rendering within a single return block for better maintainability and code readability

romiinik 2 days ago

Author

I also replaced separate conditional returns with conditional rendering in a single return block.

```
{loading && <div className="p-4 text-center">Loading charts...</div>}

{error && <div className="p-4 text-center text-red-500">Error: {error}</div>}

{!loading && !error && (
.
.
.
)}
```

Reply...

maira-fulda commented 2 days ago

Hi Romina, I have added couple of comments. Kindly check them.

`Applied review feedback` `11a5805`

romiinik commented 2 days ago Author

Hi @maira-fulda !
I applied the requested changes :)

maira-fulda merged commit `1d2fe20` into `feature/vertical-prototype` yesterday Revert

Pull request successfully merged and closed

You're all set—the `feature/dashboard` branch can be safely deleted. Delete branch

Code Review #4

Feature: Landlord Questionnaire & Tenant Questionnaire
Code by: Suhaila Kondappilly Aliyar
Reviewed by: Samarth Mahajan
Links: Pull request, questionnaire.router.ts (before & after), TenantQuestionnaire.tsx (before & after), PropertyDetailsPage.tsx (before & after)

Suggestion #1

Open **Questionnaire Changes for Review** #46
Suhaila7995 wants to merge 2 commits into `feature/vertical-prototype` from `fix_questionnaire`

samarth-mahajan1929 commented yesterday

Team5-GDSD/apps/frontend/src/components/students/TenantQuestionnaire.tsx
Lines 94 to 99 in 1c9a07b

```
      setTimeout(() => {
        navigate("/");
      }, 3000);
    } else {
      setSubmitError("Error: Tenant score not found in the response.");
    }
```

- Add loading state during the 3-second delay
- So you can show on the screen

Your response has been recorded!

Redirecting to homepage in 3 seconds...

Updated Code after Review #1

Team5-GDSD / apps / frontend / src / components / students / TenantQuestionnaire.tsx ↑ Top

Code | Blame | 192 lines (170 loc) · 5.87 KB | Code 55% faster with GitHub Copilot | Raw

```
  const handleSubmit = async () => {
    } {
      setTenantScore(result.tenantQuestionnaire.tenantScore);
      setIsRedirecting(true);
      setTimeout(() => {
        navigate("/");
      }, 3000);
    } else {
      setSubmitError("Error: Tenant score not found in the response.");
    }
  } catch (error) {
    setSubmitError("An error occurred while submitting the questionnaire.");
  } finally {
    setIsSubmitting(false);
  }
};
```

87 apps/frontend/src/components/students/TenantQuestionnaire.tsx

```diff
-            Your Score: {tenantScore}/{questions.reduce((acc, q) => acc + q.score, 0)}
-          </h3>
-          <p>Thank you for completing the questionnaire!</p>
-        </div>
+      {isRedirecting ? (
+        <p class="mt-6 text-center text-lg font-bold text-green-600">
+          Your response has been recorded! <br />
+          Redirecting to homepage in 3 seconds...
+        </p>
+      ) : (
+        tenantScore >= 0 && (
+          <div class="mt-6 text-center text-green-600">
+            <h3 class="text-lg font-semibold">
+              Your Score: {tenantScore}/
+              {questions.reduce((acc, q) => acc + q.score, 0)}
+            </h3>
+            <p>Thank you for completing the questionnaire!</p>
+          </div>
+        )
      )}
    </div>
  );
```

Suggestion #2

Open **Questionnaire Changes for Review** #46

Suhaila7995 wants to merge 2 commits into `feature/vertical-prototype` from `fix_questionnaire`

samarth-mahajan1929 commented 20 hours ago

Team5-GDSD/apps/backend/src/routers/questionnaire.router.ts
Lines 11 to 28 in 1c9a07b

```
11    router.post('/landlord/create', authenticateToken, async (req: Request, res: Response) => {
12      console.log(req.body);
13
14      const {id, name, creationDate, landlordId, questions } = req.body;
15
16      const questionnaire = DB.em.create(LandlordQuestionnaire, {
17        id:id,
18        name:name,
19        creationDate: creationDate,
20        landlordId: landlordId,
21        questions:questions
22
```

-Wrap the logic in a try-catch block to handle potential database errors consistently.

Updated Code after Review #2

Team5-GDSD / apps / backend / src / routers / **questionnaire.router.ts**

Code | Blame | 248 lines (214 loc) · 6.29 KB | Code 55% faster with GitHub Copilot

```
router.post(
  "/landlord/create",
  authenticateToken,
  async (req: Request, res: Response) => {
    console.log(req.body);

    const { id, name, creationDate, landlordId, questions } = req.body;
    try {
      const questionnaire = DB.em.create(LandlordQuestionnaire, {
        id: id,
        name: name,
        creationDate: creationDate,
        landlordId: landlordId,
        questions: questions,
      });

      await DB.em.persistAndFlush(questionnaire);

      res.status(201).json(questionnaire);
    } catch (error) {
      console.error("Error creating questionnaire:", error);
      res
        .status(500)
        .json({ message: "An error occurred while creating questionnaires." });
    }
  }
);

```

Suggestion #3

Open **Questionnaire Changes for Review** #46

Suhaila7995 wants to merge 3 commits into `feature/vertical-prototype` from `fix_questionnaire`

samarth-mahajan1929 commented now

Team5-GDSD/apps/frontend/src/components/students/PropertyDetailsPage.tsx
Lines 315 to 317 in 9d8cc07

```
315     {/* Only show questionnaire section if user is not a landlord */}
316     {!isLandlord && mode === 'user' && listing.landlordQuestionnaireId?.trim() && (
317         <div style={{ marginTop: "20px" }}>
```

- Add a check to ensure that the "Take Questionnaire" button is not displayed for logged-in moderators.

Updated Code after Review #3

3 apps/frontend/src/components/students/PropertyDetailsPage.tsx

```
140  140
141  141      const isLandlord = userRole === "Landlord";
     142  +   const isModerator = userRole === "Moderator";
142  143
143  144      const role = localStorage.getItem("userRole");
144  145      //console.log(role);

@@ -313,7 +314,7 @@ const PropertyDetailsPage: React.FC<PropertyDetailsPageProps> = ({ mode = "user"

313  314                  </div>
314  315
315  316                  {/* Only show questionnaire section if user is not a landlord */}
316     -                 {!isLandlord && mode === 'user' && listing.landlordQuestionnaireId?.trim() && (
     317  +             {!isLandlord && !isModerator && listing.landlordQuestionnaireId?.trim() && (
317  318                      <div style={{ marginTop: "20px" }}>
318  319                          <h3>Interested in this property?</h3>
319  320                          <p>
```

Re-reviewed and Merged

Open **Questionnaire Changes for Review** #46

Suhaila7995 wants to merge 4 commits into `feature/vertical-prototype` from `fix_questionnaire`

Applied Review Feedback 2f81cbe

Suhaila7995 commented 2 minutes ago Author

@samarth-mahajan1929 I have made all the changes as per the 3 reviews.

1. I have added the loading state for the questionnaire completion
2. Included try catch block for the questionnaire api
3. Added the check for not moderator as it is only available for students

Could you verify and approve my PR.

samarth-mahajan1929 merged commit `6ffdf40` into `feature/vertical-prototype` now

samarth-mahajan1929 closed this now

Pull request successfully merged and closed

You're all set—the `fix_questionnaire` branch can be safely deleted.

Delete branch

Code Review #5

Feature: Save Search
Code by: Rashid Hussain Mohammed
Reviewed by: Romina Nikasa
Links: [Pull request](), Save Search.

Suggestion #1

apps/backend/src/routers/searchHistory.router.ts Hide resolved

Comment on lines +101 to +107

```
+        const searchHistory = await DB.em.find(
+            SearchHistoryV2,
+            { user: currentUser },
+            {
+                orderBy: { creationDate: "DESC" },
+                limit: 5
+            }
```

romiinik 2 days ago

The find method is awaited, but there is no error handling for potential query failures.

Rashid-hussain-mohammed yesterday Author

But we are putting the code in the try-and-catch method which will catch all the errors that come from the try block and display the error according to the status code.

Reply...

Unresolve conversation **Rashid-hussain-mohammed** marked this conversation as resolved.

Suggestion #2

apps/backend/src/routers/searchHistory.router.ts Outdated

Hide resolved

```
+            arePetsAllowed: arePetsAllowed !== undefined ? arePetsAllowed : false,
+            searchPreferencesId,
+            amenitiesValues: amenitiesValues || [],
+            creationDate: new Date()
```

romiinik 2 days ago

The creationDate field is already initialized with a default value in the SearchHistory entity. Assigning it again in the router is redundant and can be removed to simplify the code.

Rashid-hussain-mohammed 19 hours ago • edited

Author

fixed according to the change requested

```
const searchHistory = DB.em.create(SearchHistoryV2, {
    user: currentUser,
    query,
    location: location ?? "", // Using nullish coalescing for clarity
    type: type ?? PropertyType.House, // Using enum instead of hardcoded string
    status: status ?? PropertyStatus.Pending, // Using enum for status
    maxRent: maxRent ?? null, // Null indicates "no value provided"
    minRent: minRent ?? null,
    deposit: deposit ?? null,
    availabilityFrom: availabilityFrom ?? null, //  Avoiding setting new Date() if not provic
    availabilityTo: availabilityTo ?? null,
    size: size ?? null,
    arePetsAllowed: arePetsAllowed ?? false, //  Default to false explicitly
    searchPreferencesId,
    amenitiesValues: amenitiesValues ?? []
})
```

Reply...

Unresolve conversation

Rashid-hussain-mohammed marked this conversation as resolved.

Suggestion #3

apps/backend/src/routers/searchHistory.router.ts Outdated

Hide resolved

Comment on lines 51 to 68

```
51 +         // Create new search history if no duplicate exists
```

Feature/updated save search #47

romiinik merged 12 commits into `feature/vertical-protot…` from `feature/updated_save_se…` 2 hours ago

```
55 +             location: location || "",
56 +             type: type || "General",
57 +             maxRent: maxRent !== undefined ? maxRent : 0,
58 +             status: status || "Pending",
59 +             minRent: minRent !== undefined ? minRent : 0,
60 +             deposit: deposit !== undefined ? deposit : 0,
61 +             availabilityFrom: availabilityFrom || new Date(),
62 +             availabilityTo: availabilityTo || new Date(),
63 +             size: size !== undefined ? size : 0,
64 +             arePetsAllowed: arePetsAllowed !== undefined ? arePetsAllowed : false,
65 +             searchPreferencesId,
66 +             amenitiesValues: amenitiesValues || [],
67 +             creationDate: new Date()
68 +         })
```

romiinik 2 days ago

You should set the default values for type and status using PropertyType.House and PropertyStatus.Pending instead of writing the strings directly.
And also the code mixes || and !== undefined checks inconsistently. This makes the logic harder to read and maintain.
Additionally, some default values (e.g., 0 for maxRent, minRent, deposit, and size) may not be appropriate for all use cases.
And i am also confused why we are putting some default values, for example for size.

Rashid-hussain-mohammed 20 hours ago • edited Author

I have made these changes which would reflect standard practice.

```
const searchHistory = DB.em.create(SearchHistoryV2, {
    user: currentUser,
    query,
    location: location ?? "", // Using nullish coalescing for clarity
    type: type ?? PropertyType.House, // Using enum instead of hardcoded string
    status: status ?? PropertyStatus.Pending, // Using enum for status
    maxRent: maxRent ?? null, // Null indicates "no value provided"
    minRent: minRent ?? null,
    deposit: deposit ?? null,
    availabilityFrom: availabilityFrom ?? null, //  Avoiding setting new Date() if not provi
    availabilityTo: availabilityTo ?? null,
    size: size ?? null,
    arePetsAllowed: arePetsAllowed ?? false, //  Default to false explicitly
    searchPreferencesId,
    amenitiesValues: amenitiesValues ?? []
})
```

Reply...

Unresolve conversation

Rashid-hussain-mohammed marked this conversation as resolved.

Code Review #6

Update to the add questionnaire Page
Code by Anmoldeep, reviewed by Tim Schröder
Links: Pull request, chat.router.ts (before & after)

fdai7674 commented 2 days ago

I am trying to refactor the code for removing duplicates in Tailwindcss. Is it a good approach? If so i will continue to refactor other files like this. Can u please review this approach ??

Thanks

Refactoring the code to remove duplicates 93bdfa7

fdai7674 requested a review from **ts4321** 2 days ago

ts4321 commented 2 days ago

Sounds like a good idea to remove duplicate code and make the files more readable!

Suggestion #1

ts4321 commented 2 days ago • edited

Looks like this part is missing:

Team5-GDSD/apps/frontend/src/components/Landlord/addQuestionnaire.tsx
Line 128 in 93bdfa7

```
128     className="px-4 py-2 text-white transition bg-black rounded-lg hover:bg-yellow-600"
```

Edit: resolved!

Resulting Code: Styling was added to the missing button

```diff
@@ -125,7 +127,7 @@ const AddQuestionnairePage = () => {
125 127                <button
126 128                  type="button"
127 129                  onClick={() => addOption(questionIndex)}
128     -                class="px-4 py-2 text-white transition bg-black rounded-lg hover:bg-yellow-600"
    130 +                class={styles.button}
129 131                >
130 132                  +
131 133                </button>
```

Suggestion #2

ts4321 requested changes 2 days ago

View reviewed changes

apps/frontend/src/components/Landlord/addQuestionnaire.tsx Outdated

Show resolved

apps/frontend/src/components/Landlord/addQuestionnaire.tsx Outdated

```
181  184          >
182  185            Add Question
183  186          </button>
184  187          <button
185  188            type="button"
186  189            onClick={handleSubmit}
187  -              className="px-6 py-3 text-white transition bg-black rounded-lg hover:bg-yellow-600"
     190 +          className={buttonStyle}
```

ts4321 2 days ago

I think the submit button could have a slightly different style (maybe the text could be bold?) since it is the most important button on the page.

fdai7674 2 days ago Author

I have changed the styling of the submit button

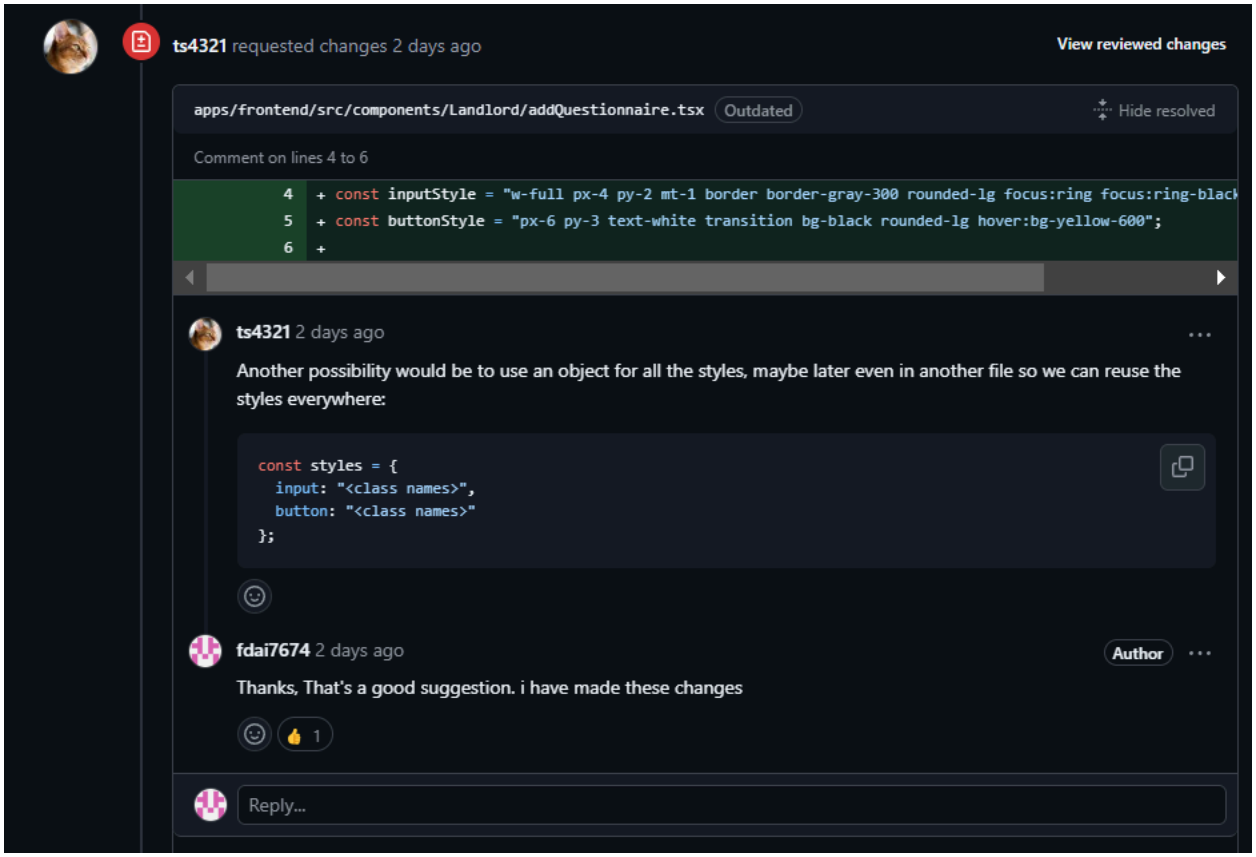
ts4321 2 days ago

Looks great!

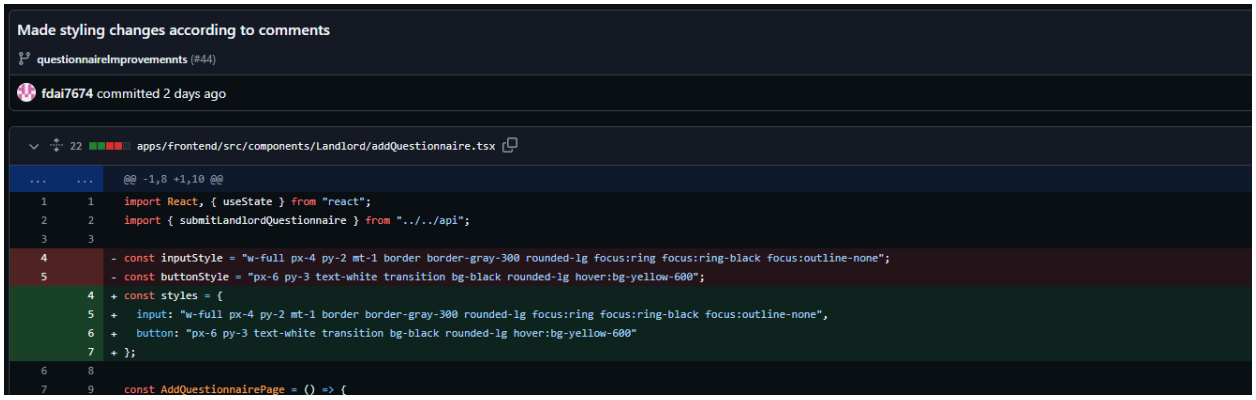
Resulting code: Made Styling changes to the submit button to stand out unique

```
188  190            type="button"
189  191            onClick={handleSubmit}
190  -              className={buttonStyle}
     192 +          className={`${styles.button} font-bold bg-blue-700`}
191  193            disabled={isSubmitting}
192  194          >
193  195            {isSubmitting ? "Submitting..." : "Submit Questionnaire"}
```

Suggestion #3



Resulting code: Made the attributes as a variable



Code Review #7

Feature:User authentication frontend & Media files upload- backend
Code by: Samarth Mahajan
Reviewed by: Rashid Hussain
Links: Pull request, media.router.ts (before & after), userAuth.tsx (before & after)

Suggestion #1

Rashid-hussain-mohammed commented 7 hours ago

Team5-GDSD/apps/backend/src/routers/media.router.ts
Lines 23 to 33 in d6e2bb8

```
23      if (files.file?.length != 1) {
24          res.status(400)
25          return
26      }
27
28      const file = files.file[0]!
29
30      if (file.size > MAX_FILE_SIZE) {
31          res.status(400).json({ error: `Maximum file size (${MAX_FILE_SIZE}) exceeded: ${file.size}` })
32          return
33      }
```

Inconsistent Naming Scheme for Error Messages, The error messages lack consistency in structure and wording.
Some messages do not explain the issue clearly. Ensure all error messages follow a uniform format and provide useful details.

Resulting code: Improved the structure and wording of error messages.

```
28  30          if (file.size > MAX_FILE_SIZE) {
29  -               res.status(400).json({ error: `Maximum file size (${MAX_FILE_SIZE}) exceeded: ${file.size}` })
    31  +           const maxSizeMB = MAX_FILE_SIZE / (1024 * 1024) // Convert to MB
    32  +           const fileSizeMB = (file.size / (1024 * 1024)).toFixed(2) // Convert to MB with 2 decimal places
    33  +           res.status(400).json({
    34  +               error: `File size limit exceeded. File size: ${fileSizeMB} MB. Maximum allowed: ${maxSizeMB} MB`
    35  +           })
30  36              return
31  37          }
```

Suggestion #2

Rashid-hussain-mohammed commented 7 hours ago

Team5-GDSD/apps/frontend/src/components/userAuth.tsx
Lines 70 to 77 in d6e2bb8

```
70      if (
71        formData.role === "STUDENT" &&
72        !formData.email.endsWith("@informatik.hs-fulda.de")
73      ) {
74        setError(
75          "Students must use a university email ending with @informatik.hs-fulda.de"
76        );
77        return;
```

Security Checks on Email Validation

```
The email validation for students (ends with ("@informatik.hs-fulda.de")) is case-sensitive.
    A user could bypass this check by entering uppercase letters.
    Use .toLowerCase() to ensure consistent validation:
```

Resultant Code: Added .toLowerCase() to ensure consistent validation

```
     83  +
68   84          if (
69   85            formData.role === "STUDENT" &&
70       -         !formData.email.endsWith("@informatik.hs-fulda.de")
     86  +         !formData.email.toLowerCase().endsWith("@informatik.hs-fulda.de")
71   87          ) {
72       -         setError("Students must use a university email ending with @informatik.hs-fulda.de");
     88  +         setError(
     89  +           "Students must use a university email ending with @informatik.hs-fulda.de"
     90  +         );
73   91            return;
```

Suggestion #3

Rashid-hussain-mohammed commented 7 hours ago

Team5-GDSD/apps/frontend/src/components/userAuth.tsx
Lines 60 to 77 in d6e2bb8

```
60      // Perform additional validation for signup
61      if (!isLogin) {
62        if (formData.password !== formData.confirmPassword) {
63          setError("Passwords do not match");
64          return;
65        }
66        if (formData.password.length < 6) {
67          setError("Password must be at least 6 characters long");
68          return;
69        }
70        if (
71          formData.role === "STUDENT" &&
```

Security Check: Prevent Spaces in Passwords, Currently, users can create a password with spaces (" 12345 "), which can lead to login issues later if trimmed incorrectly. Trim spaces before checking password length

Resultant code: Trimmed spaces before checking password length

```
59  61        if (!isLogin) {
60    -         if (formData.password !== formData.confirmPassword) {
    62  +       // Trim spaces from password and confirmPassword
    63  +       const trimmedPassword = formData.password.trim();
    64  +       const trimmedConfirmPassword = formData.confirmPassword.trim();
    65  +
    66  +       // Check if password contains spaces
    67  +       if (formData.password.includes(" ")) {
    68  +         setError("Password cannot contain spaces");
    69  +         return;
    70  +       }
    71  +
    72  +       // Compare trimmed passwords
    73  +       if (trimmedPassword !== trimmedConfirmPassword) {
61  74            setError("Passwords do not match");
62  75            return;
63  76          }
64    -         if (formData.password.length < 6) {
    77  +
    78  +       // Check length using trimmed password
    79  +       if (trimmedPassword.length < 6) {
65  80            setError("Password must be at least 6 characters long");
66  81            return;
67  82          }
    83  +
68  84          if (
69  85            formData.role === "STUDENT" &&
70    -           !formData.email.endsWith("@informatik.hs-fulda.de")
    86  +         !formData.email.toLowerCase().endsWith("@informatik.hs-fulda.de")
71  87          ) {
72    -           setError("Students must use a university email ending with @informatik.hs-fulda.de");
    88  +         setError(
    89  +           "Students must use a university email ending with @informatik.hs-fulda.de"
    90  +         );
```

5) Self-check on best practices for security

Passwords in the database are encrypted.

Protected assets:

Name	Threats	Protection
User data	Accessing sensitive information like passwords or saved payment info (planned).	In order to access data, the user needs to be logged in. Sensitive data however cannot be accessed at all. Passwords are hashed before being stored in the database.
Property data	Other users could try to update properties or delete them.	The server will return an error when a user tries to update/delete a property that does not belong to them.
Media files	Filling up the blob storage with too much data	For file uploads, a maximum file size of 10 MB is set. The server will return an error message if this limit is exceeded.
Chat messages	Threat actors could try to get other users messages	The API endpoints use the authentication token (which includes the id of the user accessing the API) to query the database

Input validation for search:

```
router.get(
    "/",
    [
        query("query").optional().isLength({ max: 40 }).isString().withMessage("query must be a string (max 40 characters)"),
        query("location").optional().isString().withMessage("location must be a string"),
        query("minPrice").optional().isNumeric().withMessage("minPrice must be a number"),
        query("maxPrice").optional().isNumeric().withMessage("maxPrice must be a number"),
        query("propertyType").optional().isString().withMessage("propertyType must be a string"),
        query("propertyStatus").optional().isString().withMessage("Status must be an enum"),
        query("arePetsAllowed").optional().isBoolean().withMessage("arePetsAllowed must be a bool"),
        query("availabilityFrom").optional().isISO8601().withMessage("availabilityFrom must be a date"),
        query("availabilityTo").optional().isISO8601().withMessage("availabilityTo must be a date"),
        query("minSize").optional().isNumeric().withMessage("minSize must be a number"),
        query("maxSize").optional().isNumeric().withMessage("maxSize must be a number")
    ],
```

6) Adherence to original Non-functional specs

List of Non-Functional Requirements

Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in Milestone 0. Application delivery shall be from chosen cloud server.

Status: DONE

Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers

Status: DONE

All or selected application functions must render well on mobile devices

Status: DONE

Data shall be stored in the database on the team's deployment cloud server.

Status: DONE

Full resolution free media shall be downloadable directly, and full resolution media for selling shall be obtained after contacting the seller/owner

Status: DONE

No more than 50 concurrent users shall be accessing the application at any time

Status: DONE

Privacy of users shall be protected, and all privacy policies will be appropriately communicated to the users.

Status: DONE

The language used shall be English (no localization needed)

Status: DONE

Application shall be very easy to use and intuitive

Status: DONE

Application should follow established architecture patterns.

Status: DONE

Application code and its repository shall be easy to inspect and maintain

Status: DONE

No email clients shall be allowed.

Status: DONE

Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI.

Status: DONE

Site security: basic best practices shall be applied (as covered in the class) for main data items

Status: DONE

Application shall be media rich (images, video etc.). Media formats shall be standard as used in the market today

Status: DONE

Modern SE processes and practices shall be used as specified in the class, including collaborative and continuous SW development

Status: DONE

For code development and management, as well as documentation like formal milestones required in the class, each team shall use their own GitHub to be set-up by class instructors and started by each team during Milestone 0

Status: DONE

The application UI (WWW and mobile) shall prominently display the following exact text on all pages "Fulda University of Applied Sciences Software Engineering Project, Fall 2024 For Demonstration Only” at the top of the WWW page. (Important to not confuse this with a real application)

Status: DONE